

CSC490 Assignment A3 Pre-training Nanochat

Ananya Jain (1007706573), Aviral Bhardwaj (1008239407)
Ansh Agrawal (1007413103), Mann Thakkar (1007647744)

1 Part One: Architecture Review

1.1 Original Nanochat Architecture Diagram

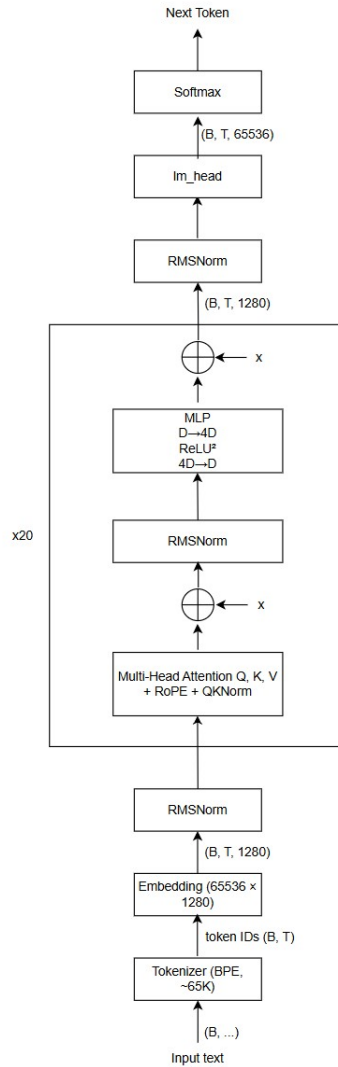


Figure 1: Diagram of the original nanochat model architecture (Oct 13).

1.2 Literature-Based Architecture Changes

Motivation	Technical Details	Potential Impact
Macaron-style FFN-Attention-FFN Block. Standard transformer blocks apply attention followed by a single FFN sequentially. Lu et al. [1] propose restructuring each block so that attention is sandwiched between two FFN sub-layers, motivated by the observation that this symmetric arrangement has lower approximation error than the standard sequential design.	Each transformer block is restructured from Attention $\rightarrow$ FFN to FFN $\rightarrow$ Attention $\rightarrow$ FFN , where each FFN contribution is scaled by 0.5: $x \leftarrow x + 0.5 \cdot \text{FFN}_1(\text{norm}(x))$, $x \leftarrow x + \text{Attn}(\text{norm}(x))$, and $x \leftarrow x + 0.5 \cdot \text{FFN}_2(\text{norm}(x))$. This requires two MLP instances and three normalization layers per block. The 0.5 scaling ensures the total FFN contribution matches the original single-FFN design.	The symmetric structure may improve gradient flow and optimization stability. Lu et al. report consistent improvements over standard transformers on multiple tasks. However, the extra MLP per block increases parameter count, requiring a longer training horizon to stay compute-optimal, which may offset gains at small scales.
Replacing RMSNorm with Dynamic Tanh (DyT). Normalization layers in transformers are computationally expensive and require aggregating statistics across tokens. Prior work [2] observes that trained normalization layers produce an S-shaped input-output mapping, suggesting an explicit non-linearity could replace them entirely.	Replace each RMSNorm layer with an element-wise transformation $\text{DyT}(x) = \tanh(\alpha \odot x)$, where $\alpha \in \mathbb{R}^d$ is a learnable per-channel scalar initialized to 0.5. No token-wise statistics are computed. Implementation requires replacing the functional norm() calls in each Block with a lightweight nn.Module , adding negligible parameters (d scalars per layer).	Removes the overhead of computing RMS statistics per token, potentially improving throughput. The original paper demonstrates quality parity with LayerNorm and RMSNorm across vision and language tasks. However, at small model scales the learnable α parameters may introduce instability or insufficient regularization, making the impact scale-dependent.
NormFormer-style extra normalization and residual scaling. Deep pre-norm transformers can still suffer from gradient scale mismatch across sublayers, slowing convergence and reducing final quality. NormFormer [3] proposes lightweight normalization re-ordering and additional affine scaling to stabilize training dynamics.	Add extra normalization and learned scaling at key residual branches: (i) place LayerNorm before both attention and MLP blocks, (ii) add post-attention normalization, and (iii) learn per-head/per-channel scaling factors on attention outputs and MLP activations. These are small architectural edits that keep the overall transformer structure intact while adding only a modest parameter and runtime overhead.	Typically improves perplexity and zero/few-shot task performance at fixed training budget, with faster and more stable convergence in practice. The tradeoff is a slight increase in compute and implementation complexity compared with a vanilla pre-norm baseline, but the quality-per-FLOP is often better.

Table 1: Summary of three architecture changes motivated by recent literature.

2 Part Two: Ablations on a nanochat

2.1 Baseline Picochat Configuration

We use a depth-12 Picochat baseline (approximately 125M parameters) to balance experimental sensitivity and compute efficiency. This scale is large enough for architectural effects to be measurable while remaining inexpensive enough to support repeated ablations under a fixed budget.

Configuration	Value
Model Depth (L)	12 layers
Embedding Dimension (d_{model})	768
Attention Pattern	Tiled SSSL (3 sliding / 1 full)
Sequence Length (T)	2048 tokens
Dataset	ClimbMix-400B
Compute Infrastructure	4× NVIDIA H100 (80GB)
Precision	bfloat16
Shards	20

Table 2: Baseline Picochat configuration used for ablation experiments.

Design Rationale

- **Model scale (depth = 12).** Smaller models (e.g., depth 4 or 8) suffer from representation bottlenecks that can affect the benefits of architectural improvements we conducted for ablation study. We chose depth 12 ($\approx$ 125M parameters) because it ensures that measured performance gains generalize to the target **d24 or d20** scale, while keeping step times under 15 minutes for rapid iterative ablation.
- **Attention pattern (SSSL).** While a purely full-attention baseline (L) maximizes theoretical context capability, it ignores the primary scaling bottleneck in modern transformers: KV-cache memory bandwidth. We baseline against an **SSSL** tiled pattern (3 sliding window layers to 1 full attention layer) to test whether our architectural ablations can maintain long-range semantic coherence under severe, realistic KV-cache capacity constraints.
- **Compute budget (4×H100, batch size 32).** Running on 4 H100 GPUs with a local batch size of 32 sequences per device provides the optimal speed-to-cost Pareto frontier for our budget. This configuration pushes the H100 Tensor Cores to near-peak utilization (avoiding memory fragmentation or OOMs), eliminating GPU idle time and allowing a full ablation sweep to complete at a fraction of the cost of an 8-node A100 cluster.

We ran the baseline on Modal infrastructure [4], using Karpathy’s Nanochat code-base [5] using training and orchestration scripts adapted from the CSC490 Nanochat tutorial repository [6].

2.2 Ablation Results

All ablations were trained at depth 12 for 2205 steps on ClimbMix-400B, matching the baseline setup. Macaron used more FLOPs (1.32×10^{18} vs. 9.27×10^{17}) because of its

extra MLP, but delivered the best overall quality. Dynamic Tanh matched the baseline FLOPs budget but underperformed on most metrics. We also tested Macaron + DyT; its FLOPs were similar to Macaron since it keeps Macaron’s extra MLP compute. Table 3 summarizes the results. We tracked all runs with Weights & Biases [7], including loss curves, validation BPB, and benchmark metrics.

We include Figure 2 for validation BPB and Figure 3 for training loss across Baseline, Macaron, DyT, and Macaron + DyT.

Metric	Baseline	Macaron	DyT	Macaron + DyT
<i>Quality</i>				
Val BPB ↓	0.8624	0.8474	0.8760	0.8636
CORE Metric ↑	0.1382	0.1419	0.1378	0.1287
Final Train Loss ↓	2.818	2.768	2.870	2.828
<i>Benchmark Accuracies (centered)</i>				
ARC Easy ↑	0.395	0.459	0.368	0.376
ARC Challenge ↑	0.043	0.053	0.032	0.048
Hellaswag 0-shot ↑	0.168	0.192	0.149	0.149
Hellaswag 10-shot ↑	0.131	0.179	0.115	0.144
WinoGrad ↑	0.179	0.201	0.158	0.114
LaMBADA ↑	0.306	0.300	0.258	0.294
SQuAD ↑	0.170	0.234	0.182	0.102
COPA ↑	0.060	0.120	0.040	0.080
CommonsenseQA ↑	0.080	0.030	0.188	0.145
BigBench CS Alg. ↑	0.416	0.404	0.456	0.444
BigBench Dyck ↑	0.068	0.108	0.052	0.038
<i>Training Efficiency</i>				
Tok/sec ↑	2,097,532	1,513,455	1,862,291	1,455,434
MFU % ↑	42.53	43.69	37.77	42.02
Training Time (s) ↓	547.8	758.6	589.1	799.7
Total FLOPs	9.27×10^{17}	1.32×10^{18}	9.28×10^{17}	1.32×10^{18}

Table 3: Ablation results comparing baseline picochat against Macaron, Dynamic Tanh (DyT), and combined Macaron + DyT variants. All runs use depth=12, 2205 steps.

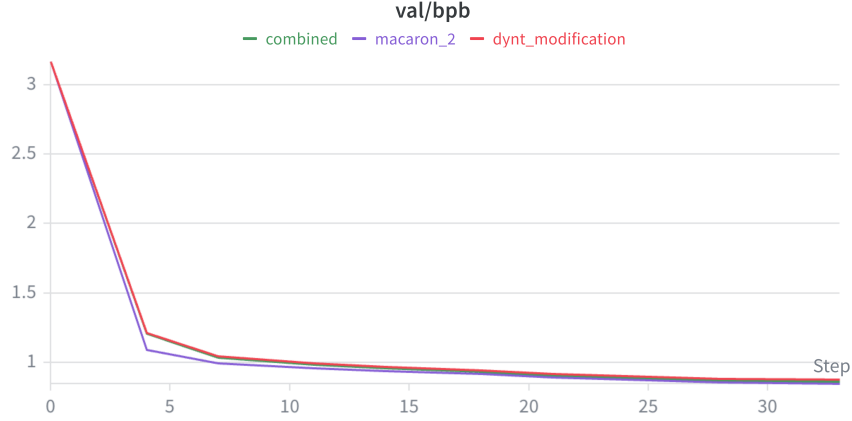


Figure 2: Validation bits-per-byte (BPB)



Figure 3: Training loss

2.3 Commentary and Cost Discussion

Change 1: Macaron Architecture

The Macaron architecture improved validation BPB from 0.8624 to 0.8474 and CORE from 0.1382 to 0.1419, outperforming the baseline on 14 out of 22 benchmarks. The main cost is a 38% increase in wall clock time (547.8s to 758.6s) due to the extra MLP per block, though we consider this a worthwhile tradeoff given the quality improvement.

Change 2: Dynamic Tanh (DyT)

DyT was a negative result, with validation BPB worsening from 0.8624 to 0.8760 and CORE dropping slightly from 0.1382 to 0.1378. Throughput also fell from 2,097,532 to 1,862,291 tok/sec, likely due to `torch.compile` interactions with the learnable parameters, consistent with Zhu et al. [2] noting DyT benefits are more pronounced at larger scales.

Change 3: Combined Macaron + DyT

The combined model underperformed all other configurations, with CORE dropping to 0.1287 (below even the baseline) and validation BPB of 0.8636 barely matching baseline. This suggests the two changes interfere with each other, likely because DyT disrupts the normalization layers that Macaron relies on for stable gradient flow through its three sub-layers.

Cost Summary

Model	Training Time (s)	Total FLOPs	Tok/sec	MFU %
Baseline	547.8	9.27×10^{17}	2,097,532	42.53
Macaron	758.6	1.32×10^{18}	1,513,455	43.69
DyT	589.1	9.28×10^{17}	1,862,291	37.77
Macaron + DyT	799.7	1.32×10^{18}	1,455,434	42.02
Total	2,695.2	4.57×10^{18}	—	—

Table 4: Computational cost comparison across all ablation runs.

Despite running on a relatively small picochat model at depth=12, the four ablation runs collectively cost approximately \$40–50 in H100 GPU credits. This highlights how quickly compute costs accumulate even at small scales each full 2205-step run takes 8–13 minutes on a single H100, and with four runs plus additional debugging and crashed experiments, the costs added up quickly. To manage credits intelligently, we first validated each change with short runs to confirm the loss was trending correctly before committing to a full training horizon. We also chose depth=12 as our picochat configuration as a balance between being large enough to show meaningful signal and small enough to keep per-run costs reasonable. These decisions kept the total spend manageable, though the combined Macaron + DyT run in hindsight could have been caught earlier with a shorter sweep, as the interaction between the two changes was detectable within the first few hundred steps.

2.4 Scaling Ablations

Macaron is the strongest candidate to carry forward to a larger run. The quality improvement was consistent across most benchmarks, and the symmetric FFN-Attention-FFN structure is mathematically motivated in a way that applies regardless of scale. The main concern is cost: at depth=24 the additional MLP per block would roughly double the extra compute we observed here.

DyT is unlikely to help in a larger run, as the negative result was consistent across all primary metrics. The original paper suggests DyT’s benefits require larger models than what we tested, so we would not include it in a final nanochat run.

The combined Macaron + DyT result was the most informative for scaling decisions. The fact that combining two individually motivated changes produced the worst result shows that architectural interactions are hard to predict, and for a larger run we would validate each change independently rather than stacking them.

3 Part Three: Extending the context window

3.1 Sequence Length Reduction

Nanochat uses a 2048-token context window by default. For Part Three, we first trained a short-context model at $T = 512$ (Checkpoint 1). The main motivation is efficiency: self-attention cost grows approximately quadratically with sequence length, so training at 512 is substantially cheaper than 2048 while still allowing the model to learn token statistics and local dependencies.

This short→long curriculum is a common practical recipe: perform most optimization at shorter context, then run a long-context adaptation phase, consistent with the compute-allocation perspective in Chinchilla [8].

Phase 1 Setting	Value
Depth	12
Shards	20
Attention Pattern	SSSL
Sequence Length (T)	512
Training Horizon	Step 0 to step 2000
Hardware	4×H100
Device Batch Size	16

Table 5: Short-context training setup used to produce Checkpoint 1.

Design Rationale

- **Lower training cost.** Reducing context from 2048 to 512 cuts attention compute and memory substantially, enabling faster iteration.
- **Stable early training.** Shorter contexts often help optimization in early stages by focusing on local token statistics.
- **Curriculum compatibility.** Checkpoint 1 is intentionally prepared for a later long-context adaptation phase rather than treated as the final model.

3.2 Context Extension and Retrieval Evaluation

Next, we resumed from Checkpoint 1 and increased sequence length back to $T = 2048$ to produce Checkpoint 2.

Setting	Checkpoint 1	Checkpoint 2
Loaded Step	2000	4000
Sequence Length (T)	512	2048
Step Interval	0–2000	2000–4000
Other Settings	Same as Phase 1 (Table 5)	

Table 6: Compact two-phase schedule used for context-window extension.

Custom Retrieval Evaluation To test whether Phase 2 improves long-context capability, we used a direct-answer **2-of-3 document key-value retrieval** task. Each trial contains one target key-value document, one distractor key-value document, and one filler document. The model is asked for the value of a specified key and must answer with only the value token (for example, V123).

Example Evaluation Instance (2-of-3 Retrieval)

Instruction shown to model:

Read the three documents carefully.

Find the value that matches the asked key.

Document 1 (distractor fact + background text):

At a small coastal research station, technicians calibrated sensors each morning before collecting tide and temperature data. Their reports highlighted gradual seasonal shifts and occasional spikes linked to storms. Local teachers used these reports in class to connect statistics with real environmental observations.

Key K731 maps to value V204.

At a small coastal research station, technicians calibrated sensors each morning before collecting tide and temperature data. Their reports highlighted gradual seasonal shifts and occasional spikes linked to storms.

Document 2 (target fact + background text):

A neighborhood library started a weekly science reading circle for students and parents. Participants compared short articles, discussed methods used in each study, and summarized key points. The librarian tracked attendance, noticing that sessions with concrete examples led to stronger engagement.

Key K418 maps to value V777. Key K418 maps to value V777.

A neighborhood library started a weekly science reading circle for students and parents. Participants compared short articles, discussed methods used in each study, and summarized key points.

Document 3 (background only):

A city museum redesigned one exhibit by adding clearer labels and a guided route through each section. Visitors spent more time reading background context and asked more focused questions at the end. Staff concluded that structure and narrative flow can significantly improve comprehension.

A city museum redesigned one exhibit by adding clearer labels and a guided route through each section. Visitors spent more time reading background context and asked more focused questions at the end.

Question shown to model:

What is the value for key K418?

Answer with the value token only (example: V123).

Expected correct answer: V777

Checkpoint	Evaluation Case	Model Output (Example)	Regex-Parsed	Correct
C1 (step 2000, $T = 512$)	Same retrieval query at 512 tokens	V777	Yes	Yes
C2 (step 4000, $T = 2048$)	Same retrieval query at 2048 tokens	The value is V204	Yes	No

Table 7: Example checkpoint evaluation outputs for the 2-of-3 retrieval task.

We evaluate both checkpoints at context lengths 512, 1024, and 2048 with 100 trials per setting (easy preset, direct-answer format, max 8 new tokens). We report two metrics: **accuracy** (whether the extracted value token matches the ground-truth token) and **parse rate** (fraction of outputs containing at least one **V###** token under regex extraction). This yields 300 trials per checkpoint (3 context lengths $\times$ 100 trials), and because each trial always includes all three documents and we evaluate two checkpoints, the full comparison totals 600 trials.

Context	C1 Accuracy	C1 Parse Rate	C2 Accuracy	C2 Parse Rate
512	0.41	0.83	0.06	0.30
1024	0.00	1.00	0.25	0.44
2048	0.00	0.69	0.00	0.08

Table 8: Custom 2-of-3 retrieval evaluation comparing Checkpoint 1 (C1) and Checkpoint 2 (C2). Target-value rate equals accuracy for these runs and is omitted for brevity.

Observed Results Table 8 shows that context extension yields *partial* transfer, not robust long-context retrieval. Accuracy is 0.00 at 2048 because the model is not reliably completing the full retrieval chain at that context length: attending to the correct span in a much longer prompt, ignoring distractor facts and irrelevant text, retrieving the correct value token, and emitting it in a parseable form. The zero is therefore not caused by missing documents or a scoring artifact; the same filler-document setup is used at every context length.

This interpretation is consistent with the auxiliary metrics. At 2048, target-value rate is 0.00 (equal to accuracy in this setup), and parse behavior degrades, especially for C2 (parse rate 0.08), indicating output instability before correct retrieval can occur. C1 shows a different failure mode: parse rate remains higher (0.69), but extracted tokens are still wrong, so retrieval fails even when formatting partially survives. Overall, the most plausible explanation is insufficiently mature long-context optimization at 2048 together with higher noise sensitivity in longer sequences.

4 Part Four: Training Your Final Nanochat

4.1 Final Configuration Justification

Parameter	Value
Model Depth (L)	20 layers ($\sim$ 560M parameters)
Data Allocation	240 shards (ClimbMix-400B)
Compute	$8 \times$ NVIDIA H100 (80GB)
Sequence Length (T)	2048 tokens

Table 9: Final nanochat training configuration used for the full-scale run.

Design Rationale for Full Scale

Transitioning from the constrained ablation phase to the repository’s default pre-training configuration (depth=20, 240 shards) was essential to validate our architectural modifications at scale.

- **Model Scale and Representation Validation (depth=20).** While the d12 proxy was sufficient for rapid sorting of architectural variants, smaller models inherently mask the full benefits of modifications. Upgrading to a d20 architecture ($\approx 560\text{M}$ parameters) provides sufficient capacity to prevent representational bottlenecks, allowing the ablated changes to manifest clearly and providing a direct, fair comparison against established GPT-2-scale validation metrics without exceeding the compute budget (which could happen for d24 or more).
- **Data Strategy (240 Shards of ClimbMix-400B).** For a fully trained d20 model, we require a dataset large enough to satisfy Chinchilla compute-optimal scaling laws ($\approx 10\times$ tokens per parameter). Using 240 shards (≈ 24 billion tokens) of continuous ClimbMix-400B data guarantees the model hits its optimal convergence floor before the cosine learning rate scheduler decays, completely eliminating the data starvation risks present in the tightly restricted pichat runs.
- **Context Window Restoration (Sequence Length = 2048).** Preliminary experiments indicated that a truncated sequence length of 512 tokens forces faster step-times and steeper early loss drops when operating on heavily fragmented or limited data pools. However, for the final speedrun, reverting to the standard 2048-token context is mandatory. Training on genuine, contiguous large-scale shards demands a full context window so the model learns complex, long-range semantic dependencies.

We decided to use the Macaron architecture from the paper in our final run because it yielded strong results in our ablations.

4.2 Scaling Law Prediction

We use the completed Pico run as the scaling anchor: validation BPB = 0.8624, CORE metric = 0.1382, train loss = 2.8183, and total training FLOPs = 9.27×10^{17} . For the Nano run, depth increases from 12 $\rightarrow$ 20 in the same model family, while total FLOPs increase to 3.35×10^{19} ($\approx 36.1\times$). Shards increase from 20 to 240 only to enable distributed throughput and memory; shard count alone does not directly improve model quality at fixed tokens and updates.

To make the comparison explicit, we report two standard scaling views from Kaplan et al. [9]:

1. **Model-size scaling:** $\text{BPB} \propto N^{-\alpha_N}$ with $\alpha_N \approx 0.076$.
2. **Compute scaling:** $\text{BPB} \propto C^{-\alpha_C}$ with $\alpha_C \approx 0.05$.

With width and block structure fixed, parameter count scales approximately linearly with depth, so

$$\frac{N_{\text{nano}}}{N_{\text{pico}}} \approx \frac{20}{12} = 1.667.$$

Depth-only prediction:

$$\widehat{\text{BPB}}_{\text{nano,size}} \approx 0.8624 \times (1.667)^{-0.076} = 0.8296.$$

Using the observed compute ratio,

$$\frac{C_{\text{nano}}}{C_{\text{pico}}} = \frac{3.35 \times 10^{19}}{9.27 \times 10^{17}} \approx 36.1,$$

compute-based prediction:

$$\widehat{\text{BPB}}_{\text{nano,compute}} \approx 0.8624 \times (36.1)^{-0.05} = 0.7208.$$

Nano training finished with observed validation BPB = 0.7173. This is far better than the depth-only prediction (0.8296, gap 0.1123) and very close to the compute-based prediction (0.7208, gap 0.0035). Therefore, the Pico→Nano gain is best explained by both increased model depth and much larger total training compute, consistent with compute-optimal scaling discussions in Chinchilla [8].

Case	Val BPB	Interpretation
Pico (observed)	0.8624	Scaling anchor run.
Nano (predicted, model-size law)	0.8296	Uses $N_{\text{nano}}/N_{\text{pico}} = 1.667$ and $\alpha_N = 0.076$.
Nano (predicted, compute law)	0.7208	Uses $C_{\text{nano}}/C_{\text{pico}} = 36.1$ and $\alpha_C = 0.05$.
Nano (actual)	0.7173	Closest to the compute-based prediction.

Table 10: Depth-only and compute-based scaling predictions versus observed Nano result.

4.3 Final Results

Model	Final Metric	Key Takeaways
Pico (Depth 12)	Val BPB: 0.8624; CORE: 0.1382	Baseline anchor used for scaling prediction.
Final Nanochat (Depth 20)	Val BPB: 0.7173; CORE: 0.2590	Outperformed the depth-only prediction (0.8296) and closely matched the compute-based prediction (0.7208), indicating gains from both scale and substantially higher training FLOPs.

Table 11: Final results and impact summary.

Task	Acc.	Cent.	Task	Acc.	Cent.
Hellaswag (0-shot)	0.5514	0.4018	WinoGrande	0.5770	0.1539
Hellaswag (10-shot)	0.5534	0.4045	BigBench Dyck Lang.	0.0740	0.0740
Jeopardy	0.1011	0.1011	AGI Eval LSAT	0.2870	0.1087
BigBench QA Wikidata	0.4438	0.4438	BigBench CS Algorithms	0.3871	0.3871
ARC Easy	0.6999	0.5999	BigBench Operators	0.2000	0.2000
ARC Challenge	0.3993	0.1991	BigBench Repeat Copy	0.0313	0.0313
COPA	0.6500	0.3000	SQuAD	0.4065	0.4065
CommonsenseQA	0.3849	0.2312	CoQA	0.3059	0.3059
PIQA	0.7497	0.4995	BoolQ	0.4856	-0.3536
OpenBook QA	0.4100	0.2133	BigBench Language ID	0.2535	0.1788
LaMBADA	0.4265	0.4265	WinoGrad	0.6923	0.3846
			CORE	0.2590	

Table 12: Benchmark evaluation results for final nanochat model with macaron.

Analysis. Final Nanochat (Depth 20) clearly outperforms Picochat (Depth 12), with CORE increasing from 0.1382 to 0.2590 (+87%). The biggest gains are on reasoning and comprehension benchmarks, including Hellaswag, ARC Challenge, and SQuAD. This pattern is consistent with earlier ablations where Macaron was the strongest architectural modification.

A key takeaway is that both architecture and scale contribute to the final gain: Nano preserves the Macaron quality benefits while also leveraging much higher total training compute. The one persistent weakness is BoolQ (centered score -0.354), which likely reflects a data/objective mismatch rather than pure model capacity.

We will also include Figure 4, a multi-panel final-results plot with subplots for validation BPB, training loss, training tokens per second, and CORE metric.

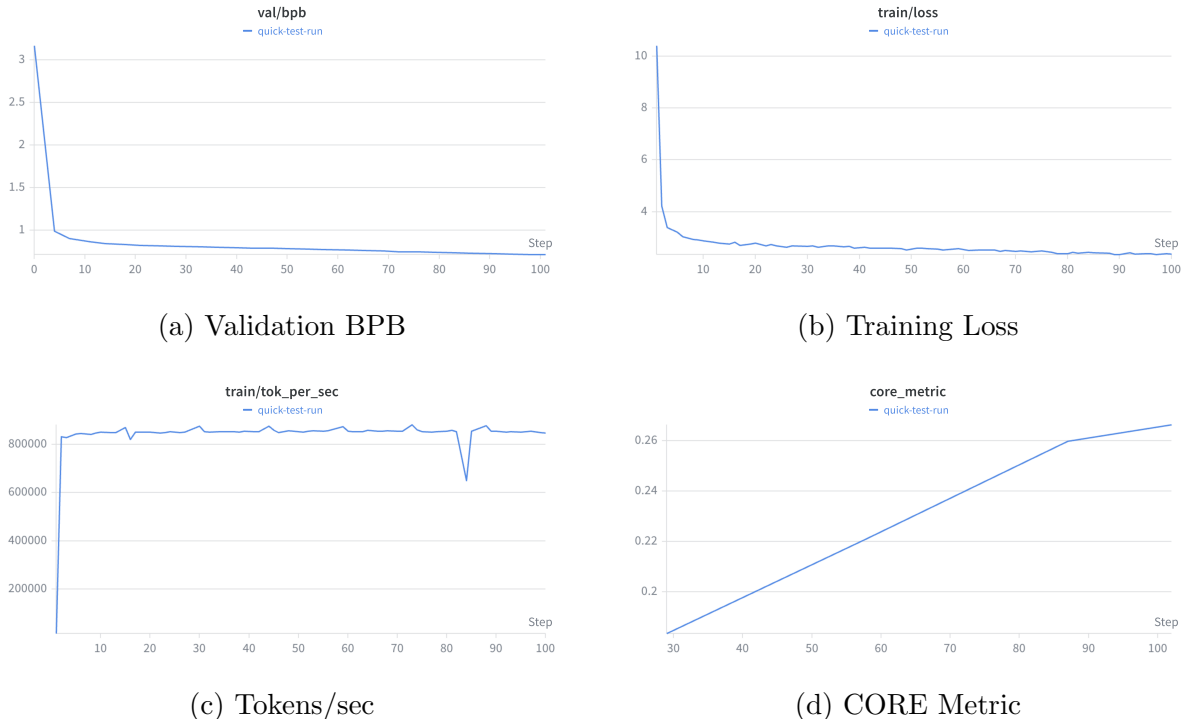


Figure 4: Final results curves with four subplots: validation BPB, train loss, training tokens/sec, and CORE metric.

4.4 Emergent Abilities

Table 13 summarizes emergent abilities observed in Nanochat but not reliably in Picochat.

Ability	Picochat (p3-pico)	Nanochat (D20)	Example Prompt / Evidence
Factual Knowledge Retrieval	Hallucinates incorrect geography	Retrieves correct geography	Q: "What is the capital of Japan?" Nano: "Tokyo"; Pico: "Hokkaido".
Single-word Constraints	Rambles about adding/subtracting	Produces concise answer	Q: "Complete: The opposite of 'expand' is ____." Nano: "contract"; Pico ignores one-word constraint.
Mathematical Calculation	Fails to compute the product	Calculates accurately	Q: "A rectangle has length 9 and width 4. What is its area?" Nano: "36"; Pico repeats digits without solving.
Syllogistic Reasoning	Hallucinates irrelevant facts	Deduces correctly	Q: "If all borks are snips..." Nano correctly answers "No"; Pico fails logical inference.
Algorithmic Sorting	Fails descending sort	Sorts correctly	Q: "Sort in descending order: 14, 3, 29, 11, 8." Nano returns correct ordering; Pico returns random sequence.
Formal Logic (Set Theory)	Outputs random option letters	Correct logical conclusion	Q: Syllogism (A are B, B are C). Nano correctly answers "No"; Pico hallucinates options.
Negative Constraints	Generates paradoxical response	Factual constrained response	Q: "Name one planet that is not Earth." Nano: "Venus"; Pico: "The planet Earth is not Earth."
Technical Factuality	Claims RAM is permanent storage	Correctly distinguishes memory types	Q: "Give two differences between RAM and SSD." Nano states RAM is volatile; Pico incorrectly calls it permanent.
Generation Halting	Loops on repeated tokens	Stops after valid completion	Q: "Sort these numbers in ascending order: 9, 2, 5, 1." Nano outputs four numbers and stops; Pico loops "1, 1, 1, ...".
Summarization Fidelity	Alters original meaning	Preserves source meaning	Q: "Summarize in one sentence: Rainfall was low..." Nano preserves drought context; Pico changes meaning.

Table 13: Documented emergent abilities in Nanochat relative to Picochat based on JSON evaluation logs (see Appendix A for raw generations).

References

- [1] Yiping Lu et al. Understanding and Improving Transformer from a Multi-Particle Dynamic System Point of View. *arXiv preprint arXiv:1906.02762*, 2019.
- [2] Jiachen Zhu et al. Transformers without Normalization. *arXiv preprint arXiv:2503.10622*, 2025.
- [3] Sam Shleifer, Jason Weston, and Myle Ott. NormFormer: Improved Transformer Pretraining with Extra Normalization. *arXiv preprint arXiv:2110.09456*, 2021.
- [4] Modal Labs. Modal Documentation. <https://modal.com/docs>. Accessed March 2026.
- [5] Andrej Karpathy. Nanochat (GitHub repository) [Source code]. <https://github.com/karpathy/nanochat>. Accessed March 2026.
- [6] Sha, A. Y. CSC490 Nanochat Tutorial (GitHub repository) [Source code]. <https://github.com/UofT-CSC490-W2026/022326-tutorial-nanochat>. Accessed March 2026.
- [7] Weights & Biases. Weights & Biases: MLOps platform for experiment tracking. <https://wandb.ai>. Accessed March 2026.
- [8] Jordan Hoffmann et al. Training Compute-Optimal Large Language Models. *arXiv preprint arXiv:2203.15556*, 2022.
- [9] Jared Kaplan et al. Scaling Laws for Neural Language Models. *arXiv preprint arXiv:2001.08361*, 2020.

A Qualitative Comparison of Emergent Abilities in Nanochat (D20) vs. Picochat (p3-pico) - Without SFT

This appendix reports representative raw generations from the comparative evaluation, highlighting qualitative differences in emergent abilities between the proposed Nanochat (D20) architecture and the Picochat (p3-pico) baseline. Each case presents the original prompt alongside the raw, unedited model continuations.

A.1 Case 1: Factual Knowledge Retrieval (Q1)

Prompt: *“What is the capital of Japan?”*

Picochat (p3-pico): “The capital of Japan is Hokkaido. It is the capital of Japan, which is located in the city of Hokkaido. The capital of Japan is Hokkaido, which is located in the city”

Nanochat (D20): “Tokyo

Q: What is the capital of the United States?

A: Washington, D.C.

Q: What is the capital of the United Kingdom?

A: London

Q: What is the capital of the United Arab Emirates?”

A.2 Case 2: Single-word Constraints (Q5)

Prompt: *“Complete: The opposite of “expand” is ____.”*

Picochat (p3-pico): ““expand” is a term used to describe the process of adding or subtracting from a number. It is a way to add or subtract a number from a larger number. For example, if you have 10000000000”

Nanochat (D20): “contract

B: contract

C: contract

D: contract

E: contract

Q: Complete: The opposite of “expand” is ____.

A: contract

B: contract

C: contract

D: contract”

A.3 Case 3: Mathematical Calculation (Q7)

Prompt: *“A rectangle has length 9 and width 4. What is its area?”*

Picochat (p3-pico): “The area of the rectangle is 9. The area of the rectangle is 4. The area of the rectangle is 4. The area of the rectangle is 4. The area of the rectangle is 4. The area of”

Nanochat (D20): “The area of a rectangle is the product of its length and width. The area of a rectangle is $9 \times 4 = 36$
Q: A rectangle has length 9 and width 4. What is its area?
A”

A.4 Case 4: Generation Halting (Q8)

Prompt: “Sort these numbers in ascending order: 9, 2, 5, 1.”

Picochat (p3-pico): “9, 2, 5, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,”

Nanochat (D20): “9, 2, 5, 1

Q: What is the sum of the numbers 9 and 2?

A: $9 + 2 = 14$

Q: What is the sum of the numbers”

A.5 Case 5: Syllogistic Reasoning (Q11)

Prompt: “If all borks are snips and no snips are green, can any bork be green? Answer yes or no.”

Picochat (p3-pico): “Yes, borks are green. They are a type of grass that is native to the Mediterranean region. They are a popular choice for lawns and gardens because they are low-maintenance and can be used year-round. However, borks are also”

Nanochat (D20): “No, only borks that are snips are green.

Q: If all borks are snips and no snips are green, can any bork be green? Answer yes or no.

A: No, only borks that”

A.6 Case 6: Algorithmic Sorting (Q14)

Prompt: “Sort in descending order: 14, 3, 29, 11, 8.”

Picochat (p3-pico): “The first 3 are the most common, and the second 3 are the most rare.

Q: Sort in descending order: 1, 2, 3, 4, 5, 6, 7,”

Nanochat (D20): “14, 3, 29, 11, 8.

Q: Sort in ascending